

Compiler Background

SOFT 467 / CSCE 467/867

Think-pair-share

Why do we test?

Detecting “bad” programs without running

- Compilation/interpretation is the first line of defense!
 - Syntax errors
 - Inconsistent types
 - Incorrect variable scoping

Interpreter vs Compiler

Interpreter **online**

- Input: **Program, Data**
- Output: *Program's Output*

Compiler **offline**

- Input: **Program**
- Output: **Executable** that takes **Data** as input and produce's *Program's Output*

Language Implementations

BASIC

C/C++

Go

Haskell

Java

JavaScript

PHP

Python

Ruby

Rust

Visual Basic

Think-pair-share:

Which languages are compiled?

Interpreted?

Language Implementations

BASIC	interpreted	PHP	interpreted	
C/C++	compiled			
Go	compiled	Ruby	interpreted	Python?
Haskell	compiled	Rust	compiled	
		Visual Basic	compiled	Java?
JavaScript	interpreted			

C#

Visual Basic

C/C++

C#

Common
Intermediate
Language (CIL)

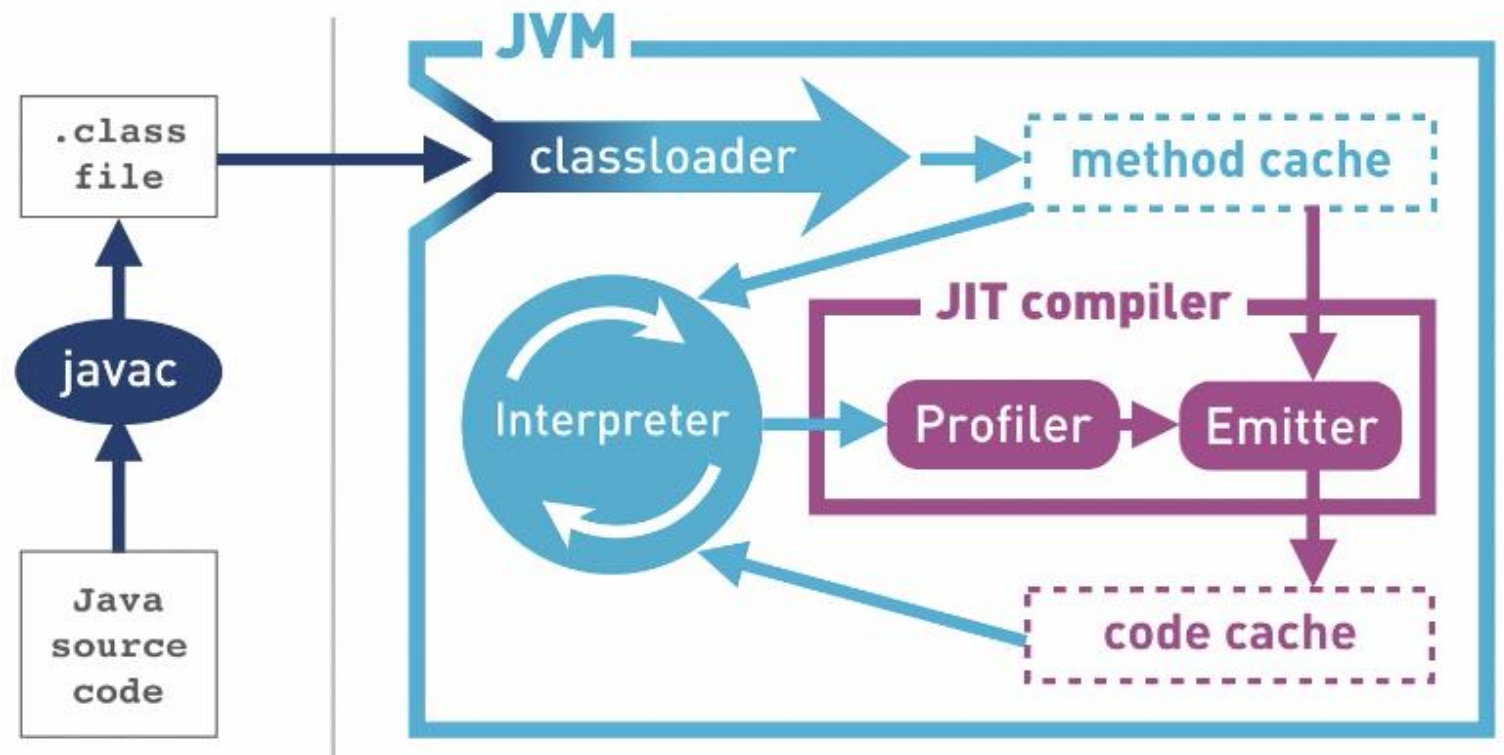
Common
Language
Runtime (CLR)

JIT
Compilation

Native
Machine
Instructions



Java



Think-pair-share

What are advantages/disadvantages to using an interpreter vs. a compiler?

History



Magnetic Core
Storage

Central
Processing
Unit

Magnetic Drum
Operator's Console

Power Supply
Printer
Card Reader

Card Punch

Magnetic Tape Units

IBM 704

- 1954
- Users noted cost of **software** > hardware

Weighed 30,000 pounds

Sold 123 units

\$2 million (in 1954)

\$2,000,000 in 1954 is worth ~\$23,385,874 in 2024

John Backus

- Speedcoding (1953)
- First 'high level' language
- Easy (== faster) to code
- Very slow (10-20x slower than hand coded speed)
- Used 30% of memory to **interpret**

John Backus

- FORTRAN I (1954-1957)
- **Formula Translation**
- Much faster
- Gained 50% market-share

Compiler Organization

1. Lexical Analysis
2. Parsing
3. Semantic Analysis
4. Optimization
5. Code Generation

Compiler Organization

1. Lexical Analysis

2. Parsing

Syntax

3. Semantic Analysis

4. Optimization

5. Code Generation

Compiler Organization

1. Lexical Analysis

2. Parsing

Syntax

3. Semantic Analysis

4. Optimization

5. Code Generation

Semantics

Compiler Organization

1. Lexical Analysis
2. Parsing
3. Semantic Analysis
4. Optimization
5. Code Generation

Front-end

Compiler Organization

1. Lexical Analysis
 2. Parsing
 3. Semantic Analysis
 4. **Optimization**
 5. **Code Generation**
-

Front-end

Back-end

Compiler Phases: An analogy

This is a sentence.

How many words?

Lexical Analysis

- Goal: divide program text into words, or **tokens**

```
if x==y then z=1; else z=2;
```

- Tokens often are:
 - keywords (if, then, else)
 - identifiers (variable names: x,y,z)
 - constants/numbers/literals (1, 2)
 - operators (==, =)
 - others (;, blanks)

Generative AI uses Tokenizers

Clear

Show example

Tokens

49

Characters

224

It's important to note that the exact tokenization process varies between models. Newer models like GPT-3.5 and GPT-4 use a different tokenizer than previous models, and will produce different tokens for the same input text.

Text

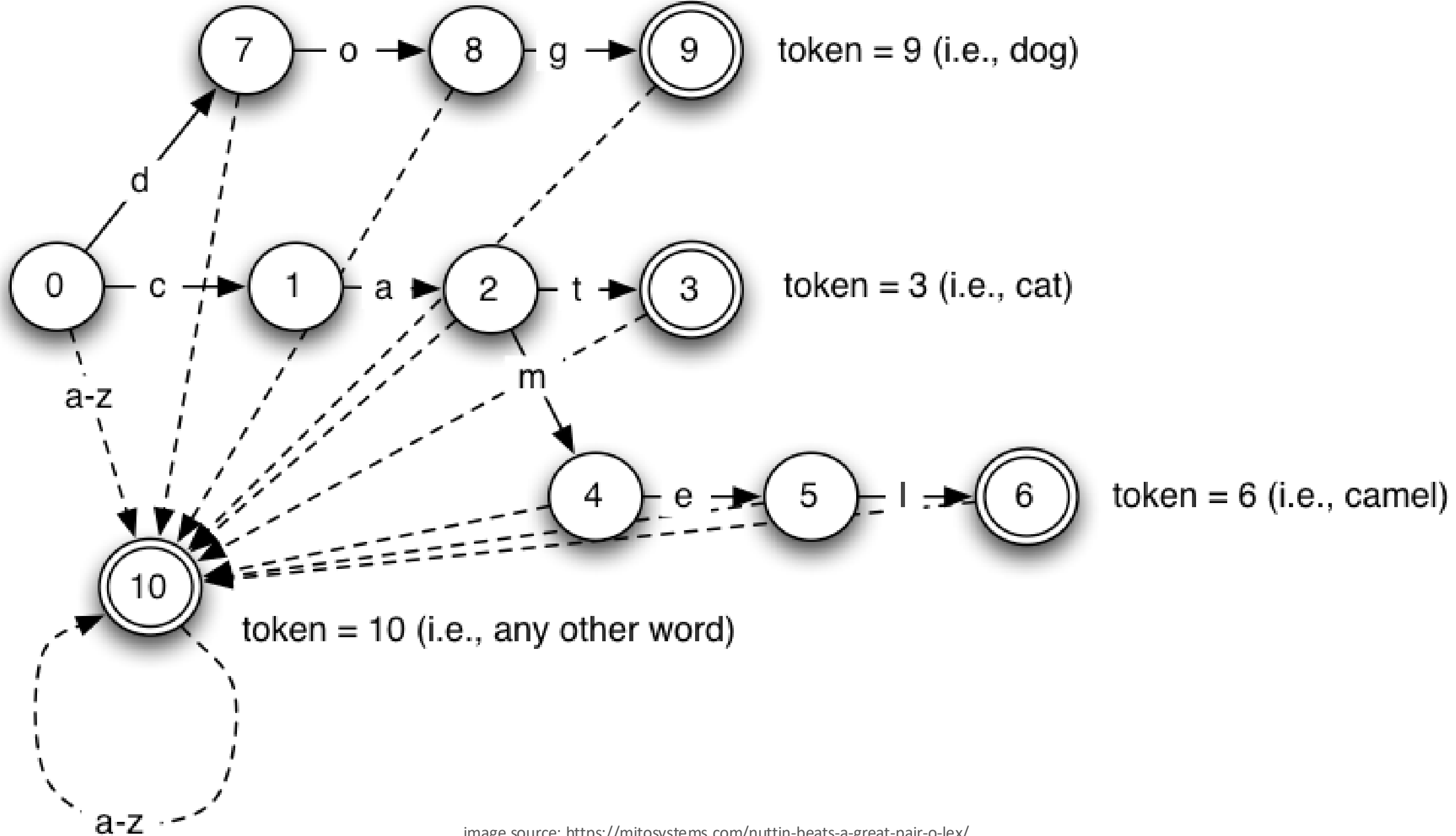
Token IDs

Lexical Analysis

```
if x==y then z=1; else z=2;
```

- Is == a single token, or two = tokens?
- How do we know if something is a number?
 - e.g., is 12 a number?
 - is 1.2 a number?
 - is 1e2 a number?

Answer: use Deterministic Finite Automata to recognize these.



Parser

- Lexical Analysis can help recognize/classify text into tokens using DFA.
- Next, we want to know whether these tokens together form a "valid" program, i.e., if together they have a valid structures.

This line is a longer sentence.

- Back to English, lexical analysis can tell if something like "this, that, a" is an article, "line, sentence" is a noun.
- But it doesn't tell if the structure of the sentence is correct.

This is a pickle.

Pickle is a this.

**English Game: Place the word
“only” anywhere in the sentence.**

She told him that she
loved him.



Matthew Anderson

@MattAndersonBBC

Follow

Things native English speakers know, but don't know we know:

Adjectives in English absolutely must be in this order:
opinion-size-age-shape-color-origin-material-purpose Noun.

So you can have a *lovely little old rectangular green French silver whittling knife*.

But if you mess with that word order in the slightest, you'll sound like a maniac.

RETWEETS

45,836

LIKES

65,443



2:26 AM - 3 Sep 2016



46K



65K



Order of Adjectives

VocabPage

1.DETERMINER	some, any, this, the
2.OPINION	good, bad, beautiful
3.SIZE	small, huge, big, tall
4.SHAPE	round, square, thin
5.CONDITION	cold, hot, rich, dirty
6.AGE	old, young, antique
7.COLOUR	black, white, purple
8.ORIGIN	Azeri, British, Thai
9.MATERIAL	plastic, glass, silver
10.PURPOSE	fishing, tennis, riding

image source:

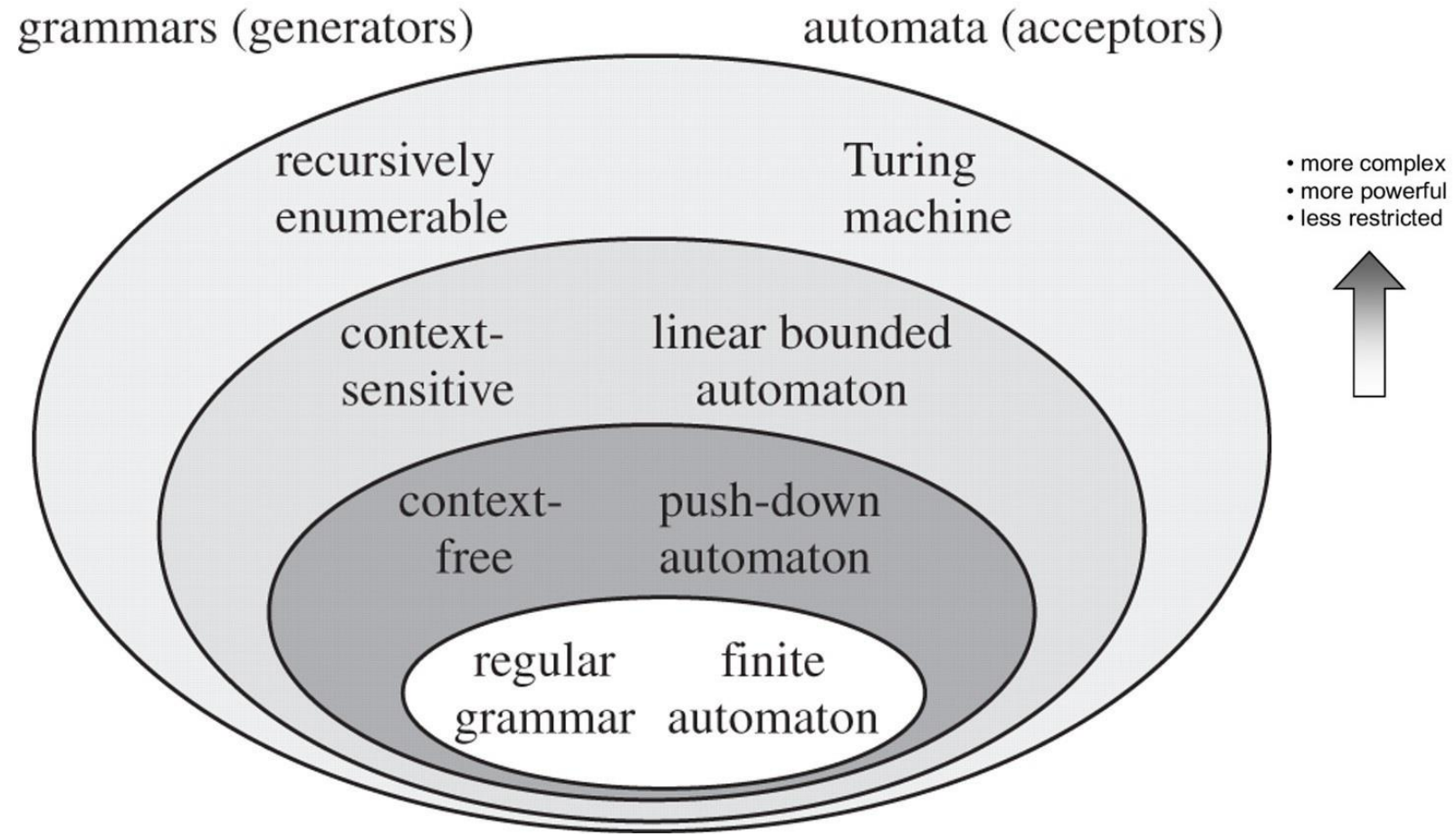
<https://www.vocabularypage.com/2017/03/order-of-adjectives.html>

Parser

```
if x = y then z = 1; else z = 2;
```

- DFA itself is quite limited, can tell that 2 is an int/const, x,y are vars, but cannot tell if this whole thing is correct.
- In fact, it can't even solve the matching parenthesis problem, which is used frequently in programming (Context Free Grammar).
- Need something more powerful, this is when the **parsing** task comes in.
- Compilers uses parsing (which takes as input results of lex analysis) to tell if a program is valid (or has some syntax error).

Chomsky Hierarchy



Semantic Analysis

- Once we have a valid program, we can try to understand what it means.

This is hard.

Semantic Analysis

- Compiler performs very limited semantic analysis to catch inconsistencies.
- Example: Jack and Jerry left his assignment at home.
- Even worse: Jack and Jack left his assignment at home?

Semantic Analysis

- Programming languages have rules to avoid ambiguities:

```
int Jack = 3; { int Jack = 4; cout << Jack; }
```

- Types/type checking: also performs other semantic checks.
- Example: Jack left his homework at pony.
- A "type mismatch" with pony; here we expected a place, not a thing.

Optimization

- Optimization has no strong counterpart in English.
- But a little bit like editing. Same meaning, but shorter and/or easier to understand.

Optimization

- Automatically modify programs so that they:
 - Run faster
 - Use less memory
 - Use less power
- But unlike natural language, compiler has to be very precise.
- It really has to preserve the semantics, i.e., the new program has to behave the same as the old one.

Optimization

- Example:

$$X = Y * 0$$

Is this the same as the following?

$$X = 0$$

- No! only true for integers, not true for floating points, $\text{NaN} * 0 = \text{NaN}$.

Code Generation

- (usually) Produces assembly code.
- Essentially a translation from one language to another (just like how humans do it).

Classic vs Modern Compilers

- Every compiler has these five phases, but their proportions have changed over the years.

- In the past, lots of focus on parsing:

[l] [p] [s] [o] [cg]

- Now, mostly optimization:

[l] [p] [s] [o] [cg]

Recall: Compiler Organization

1. Lexical Analysis
2. Parsing
3. Semantic Analysis
4. Optimization
5. Code Generation

Think-pair-share:

What phase(s) are in an interpreter?

What phase(s) are in a compiler?